СОГЛАСОВАННАЯ АРХИТЕКТУРА

Backend: контракты, данные и транзакции

Единый контракт, проверяемые границы и конкретные правила реализации.

20 октября 2026
Согласованная спецификация

Команда 3-5 человек
Окно 48-72 часа

Приложение не реализовано. Статические проверки документации и контрактов отделены от будущих испытаний runtime.

Связанные источники, контракты и приёмка доступны в Markdown-руководстве и общем index.html комплекта.

HACKATHON-1.5. Java 21, Spring Boot 4.1.1, PostgreSQL 17.11 как проверенные документальные ориентиры. Приложение не собрано. Точные JDK/OCI digests и совместимость зависимостей закрывает P02. Spring Modulith исключён из обязательного MVP: модульные границы контролируются пакетами, code review и ArchUnit после проверки его версии. Логическая модульность сохраняется.

1. Модули и поток вызовов

Access → Catalog → Applications/Consents/Documents → Journey → Bank simulator → Workspace/Offers. ML вызывается через отдельные порты Feature, Risk, Ranking, OCR и Rag. Контроллеры вызывают application services, а не repositories. JPA entities не являются HTTP DTO. Open-in-view выключен; транзакция не удерживается во время HTTP, OCR, LLM или ожидания банка.

Схемы PostgreSQL по владельцам: access, catalog, applications, consents, documents, journey, integration, workspace, ml, kb. В MVP один кластер; миграции принадлежат соответствующим модулям, порядок согласует backend-владелец. Python получает разрешённые API и минимальный доступ к своей ml-схеме, но не к чужим таблицам.

Банковский симулятор сохраняет case/offer/command-result, ведёт caseVersion и применяет ту же семантику команд, что target-adapter. Это отдельный авторитет решения внутри процесса Java, а не фронтендовая кнопка изменения статуса. Нормальный happy path автоматический; demo-оператор нужен для ручной проверки и моделируемого завершения.

2. Транзакции и команды

Синхронный create возвращает 201 и тот же созданный ресурс при повторе; не придумывать operationId для каждого 201. Долговечная команда submit/response/accept/withdraw возвращает 202 и сохранённый receipt с operationId. До HTTP-ответа DB commit уже завершён.

Порядок команды: проверить аутентификацию и принадлежность company context; найти сохранённый ключ; при совпадающем hash вернуть прежний результат; иначе проверить роль, актуальность версий и бизнес-предусловия; одним commit сохранить изменение, operation, idempotency и outbox. На новых командах полномочия проверяются повторно непосредственно перед commit. Отозванная identity не получает результат чужой операции даже по старому ключу.

Дедуп обеспечивают уникальные ограничения, а не check-then-insert без блокировки. Consumer фиксирует eventId/inbox и результат атомарно. Dispatcher использует SKIP LOCKED, ограниченный batch, backoff и leases; poison message переводится в видимую ошибку, не исчезает. Истёкший worker token/generation не может завершить новое задание.

В local-mvp outbox передаётся локальным transport тем же обработчикам. Kafka не обязательна. Case projection читает полный нормализованный поток и продвигает caseVersion для всех его типов; gap инициирует CaseSnapshot и сверку. Нельзя подписаться на часть событий и ожидать последовательные глобальные версии.

Принятие/истечение/отзыв предложения разрешает единый банковский CAS. Результат зависит от банковского времени и исходного offerVersion, а не состояния кнопки в UI. Исходная заявка и предыдущие финансовые снимки остаются неизменными. Отзыв DRAFT выполняется локальным CAS без банковского дела; после отправки результат отмены подтверждает симулятор/банк.

3. Финансовые данные и RFI

Все три demo-продукта требуют восемь подтверждённых значений DEMO Financial v1. Неполный черновик разрешён. В MVP не просить повторно вводить Business.annualRevenue: UI показывает его как предварительную диагностику, а после подтверждения авторитетен FinancialStatement.revenueMinor. Policy cap и модель используют только подтверждённый финансовый снимок. annualExpenses/currentDebt не подменяют EBIT/totalLiabilities; возраст бизнеса остаётся отдельным eligibility-полем. В финансовой форме periodStart/periodEnd задают один полный календарный год; RUB, integer minor units. Equity, netProfit, EBIT могут быть отрицательными. totalAssets >0, currentAssets ≤totalAssets, currentLiabilities ≤totalLiabilities. Пропуск и ноль различаются.

Для синтетического скорингового среза баланс проверяется с точностью исходных данных и допуском 1 копейка. Допуск не является банковской нормой. Подтверждение требует всех восьми полей, по одной уникальной provenance на поле, совпадения statementVersion и точных source document versions. Внесённое изменение сбрасывает подтверждение; сервер формирует actor/time/source. Обычный PATCH не принимает confirmed/provenance/confirmedExtractionIds.

При submit после flush UI сохраняет expectedVersion, requirementsVersion, financialStatementVersion, documentIds и documentReferences. Сервер сравнивает сохранённые версии, создаёт financialSnapshotId и immutable CreditPackage в одной транзакции. Клиент не может передать financialSnapshotId. Feature snapshot вычисляется после commit из этого финансового снимка и сохраняется неизменяемо до risk inference.

RFI имеет requestVersion, OPEN/CLOSED и адресованные FIELD/DOCUMENT/EXPLANATION. Финансовый ответ начинается полной копией прежней формы; PUT может изменить только запрошенные значения. Даже если финансовый блок не менялся, response закрепляет текущую подтверждённую версию. Подтверждение полного блока не разрешает изменять незапрошенные поля. Response сравнивает draftVersion, requestVersion, requirementsVersion и сохранённое тело, затем создаёт новый amendment/financial snapshot. SUBMITTED исходная анкета не изменяется.

При PATCH business merge выполняется по полям; отсутствие означает «не менять», разрешённый clearFields - «очистить». Присваивание и очистка одного пути одновременно даёт 422. Версии и целые числа ограничены безопасным диапазоном JSON 1..9007199254740991.

4. Файлы

Upload intent закрепляет company/application, размер, MIME, checksum, requirementId и парные requestId/requestVersion при RFI. Файл передаётся потоково с ограниченным буфером; лимит 10 MiB/5 страниц, PNG/JPEG/PDF. Сначала quarantine; проверяются сигнатура, MIME, структура и AV. Рендер страницы ограничен 40 млн пикселей; лимит декодированного изображения проверяется до выделения полного буфера. PDF с активным содержимым/вложениями/шифрованием не принимается в demo-форму; рендеринг идёт в изолированном worker с лимитом времени/памяти.

CLEAN требует подтверждённого objectVersion, checksum и положительного scanner verdict. AV ERROR остаётся quarantine. Signature DB scanner включена в подготовленный bundle, в runtime сеть для обновления не нужна; отсутствующая база не превращается в успешную проверку. Для optional-формы пользователь может удалить файл и ввести данные вручную. В демонстрации OCR используется настоящий CLEAN-файл.

Cancel upload и finalization используют один CAS upload-state. Проигравший объект удаляется сборщиком мусора; он не становится доступным. Completion повторно проверяет grant/context/размер/checksum/срок. Grant короткоживущий, связан с сессией и действием, не публикуется в логах. Download проверяет права и точную версию; никакого произвольного filesystem/S3 URL из браузера.

OCR lease содержит workerId, leaseToken, generation, job type, expiry и objectVersion. READY не меняет подтверждённые данные автоматически. Обновлённый файл или истёкшая аренда делают старый результат недействительным.

5. ML и банковское решение

Риск: POST /internal/v2/risk/evaluations получает ссылку и digest feature snapshot. GET /internal/v2/feature-snapshots/{featureSnapshotId} возвращает именно неизменяемый снимок, доступный роли ml-risk в ML-сервисе. FeatureSnapshotInput содержит financialSnapshotId/digest и точные financialStatement, скопированные адаптером из CreditPackage. Python не читает «последнюю анкету компании». Запрос с несовпадающим digest отклоняется. Inference record фиксируется до успешного ответа.

Рекомендации: backend отбирает eligibility/publication, передаёт IDs, тексты и разрешённый coarse context. Ответ проверяется как полная перестановка множества. Метаданные exposure сохраняются для MODEL и RULES. PRODUCT требует eligibilityVersion. Для feedback itemId равен productId или FeedCard.cardId; чужой exposure запрещён, clientEventId уникален в контексте пользователя.

RAG: BFF создаёт контекст companyId/subjectRef, allowedSources, kbEpoch и requestId. Внутренний API доступен только роли bff. Backend ещё раз проверяет источники после генерации и при чтении истории/ссылок. Никакие поля браузера не могут расширить allowlist.

Для synthetic-demo-policy-2 исходная DQ проверяется до submit. Расхождение выручки >20% с reference → NEEDS_INFO; после eligibility отрицательный equity или liabilities/assets≥0,9 → DECLINED; нулевая выручка и ML error → MANUAL. Затем score≥T_manual → MANUAL до расчёта альтернатив. Закреплённый порог validation: 0.03908894142283073. При допустимом риске cap=min(product.maxAmount,revenueMinor//4); cap ниже минимума → MANUAL, превышение cap → ALTERNATIVE, иначе APPROVED. Пороги искусственные, score не PD. Реализованный ML API и эталон policy: docs/ml-integration.md корня проекта; backend/UI создают интеграторы.

Каталог: оборотный 100 тыс.-5 млн RUB/3-24 мес.; линия 300 тыс.-10 млн/6-36 мес.; овердрафт 50 тыс.-2 млн/1-12 мес. Все synthetic; возраст бизнеса минимум 6/12/12 месяцев соответственно. Диагностика со сведениями UNKNOWN просит уточнить вход, а не угадывает eligibility. Предложение содержит полную сумму, срок, disclosure, version, validUntil (+24 часа demo clock). Принятие актуальной версии отличается от «подписания/выдачи».

6. Эксплуатация и проверки

Бюджеты: обычный API 2 с, risk 3 с, ranking 2 с, RAG 60 с, OCR 120 с; внешние вызовы используют отдельные ограниченные executors. Под общий web pool не ставить генерацию LLM. Метрики: latency/errors, DB pool, outbox age, jobs by state, stale projections, denied access и degraded ML modes. Logs содержат trace/opaque IDs, но не токены, файлы и полные финансовые/чат-тела.

Приёмка: docs/acceptance.md. H2 не заменяет PostgreSQL; статические проверки не заменяют runtime-нагрузку и DR.

Технологические адаптеры

SessionStorePort/CachePort разделяют Redis-сессии и восстанавливаемый кеш; EventTransportPort сохраняет PostgreSQL outbox/inbox при подключении Kafka; ObjectStorePort связывает filesystem/S3 с exact objectVersion и CLEAN. P25 реализует локальные адаптеры и composition root. Идемпотентность, CAS, полномочия и quarantine не заменяются заглушками; отказ authority закрывает действие.

Полная спецификация адаптеров; машинный каталог - contracts/technology-adapters.json из корня комплекта.